

Technical Report: L1 Data Cache Simulation and Performance Analysis

Computer Architecture Simulation Lab

February 13, 2026

1 Problem Description

In modern computing, a significant performance disparity exists between high-speed CPUs and relatively slow Main Memory (DRAM). A system where a CPU communicates directly with RAM creates a ”**Memory Wall**”, where the processor spends the majority of its cycles idling while awaiting data operands. To mitigate this latency, we implement an *L1* Cache component.

The primary advantages of this implementation include:

- **SRAM vs. DRAM:** Caches utilize Static RAM (SRAM), employing a 6-transistor flip-flop design that is significantly faster than the capacitor-based Dynamic RAM (DRAM) used in main memory.
- **Proximity:** *L1* caches are ”On-Die,” minimizing physical propagation delay compared to off-chip RAM modules.
- **Lookup Speed:** The small footprint (32 KiB) reduces electrical capacitance and simplifies tag-matching logic, enabling near-instantaneous access.

2 Proposed Solution and Design

Our solution implements a 32 KiB, 8-way set-associative *L1* D-cache with a 32-byte line size.

2.1 Cache Geometry and Tradeoffs

The cache organization is defined by the following parameters:

- **Cache Size (C):** 32,768 bytes
- **Block Size (B):** 32 bytes
- **Associativity (E):** 8 ways

The number of sets (S) is calculated as:

$$S = \frac{C}{B \times E} = \frac{32768}{32 \times 8} = 128 \text{ sets}$$

We chose an 8-way set-associative design to balance the following architectural tradeoffs:

- **Direct-Mapped ($E = 1$):** Simple but prone to high conflict misses (thrashing).
- **Fully Associative ($S = 1$):** Eliminates conflict misses but requires complex, power-hungry CAM (Content Addressable Memory) to search all slots.
- **8-Way Set-Associative:** Provides a ”middle ground,” allowing data to reside in any of the 8 ways within a set, reducing conflict misses while maintaining a 1-cycle access time.

2.2 Functional Policies

- **Write-Back Policy:** Updates to RAM occur only when a "dirty" line is evicted, reducing bus traffic.
- **Allocate-on-Write:** On a write miss, the 32-byte block is fetched from memory before the update, ensuring line atomicity.
- **Least Recently Used (LRU):** Leverages *temporal locality* by evicting the line that hasn't been accessed for the longest duration.

3 SystemC Implementation

3.1 Modular Design and Signaling

Following tutorial-0, the simulation is implemented in SystemC, modeling hardware components as classes derived from `sc_module`. The system is composed of three primary modules: the **CPU** (driver), the **Memory** (backing store), and the newly introduced **Cache** (intermediary).

Communication relies on the `sc_signal_inout_if` interface. This allows for bi-directional communication where multiple modules share the same wires. To prevent bus contention, the protocol mandates that a writer must "float" the bus (write high-impedance 'Z') after transmitting data, allowing the opposing component to drive the line.

3.2 Architectural Changes

The introduction of the L1 Cache fundamentally alters the system topology. In the previous model (Figure 1 in tutorial-0), the CPU drove the Memory ports directly. In this implementation, the Cache acts as a transparent proxy.

- **CPU Interface:** The CPU is unaware of the cache's existence. It drives **Addr**, **Func**, and **Data** exactly as it did for main memory.
- **Cache Intermediary:** The Cache intercepts these signals. If a hit occurs, it responds immediately. If a miss occurs, the Cache freezes the CPU (by holding **Done** low) and acts as a driver to the Main Memory.

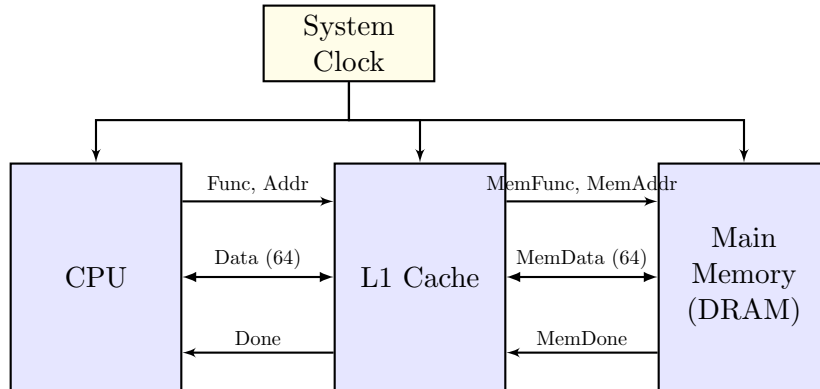


Figure 1: Updated System Architecture. The Cache isolates the CPU from the slow DRAM, handling the handshake protocol on both sides.

3.3 Updated Communication Protocol

The handshake protocol now varies based on data locality. Figure 2 illustrates the complexity of a **Read Miss**, which requires the Cache to pause the CPU, fetch a block from Memory, and then fulfill the CPU's request.

1. **Request:** CPU asserts *Addr* and *Func* (Read).
2. **Tag Check:** Cache compares tags. On a *Miss*, it suspends the CPU transaction.
3. **Fill:** Cache issues a read to Main Memory.
4. **Latency:** Memory waits 100 cycles, then drives *MemData* and asserts *MemDone*.
5. **Update:** Cache fills the line, then drives the requested word to the CPU and asserts *Done*.

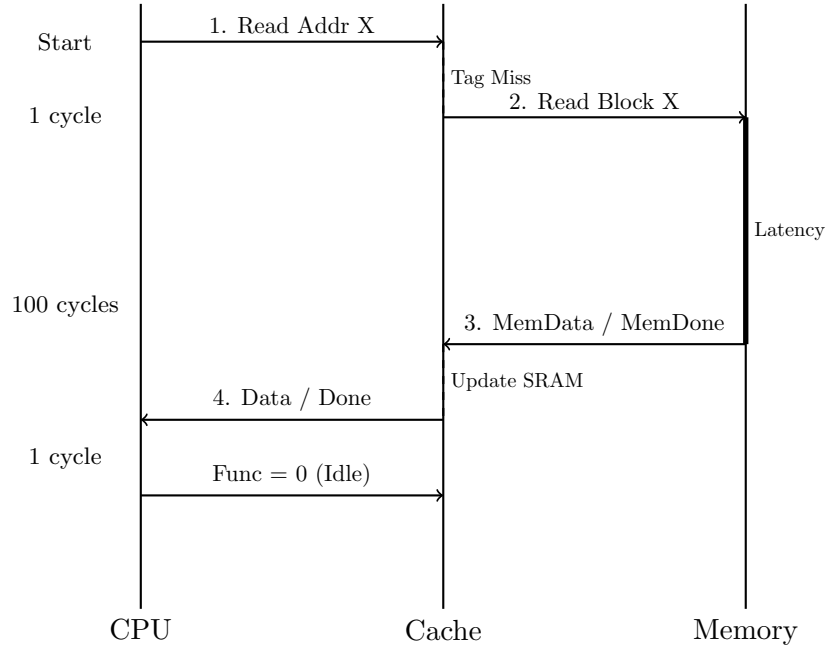


Figure 2: Sequence Diagram for a Read Miss. The Cache acts as a master to the Memory and a slave to the CPU, adding latency only when necessary.

4 Experimental Methodology

Experiments were conducted using SystemC to simulate hardware behavior with $L_{cache} = 1$ cycle and $L_{mem} = 100$ cycles.

4.1 Spatial Locality

Evaluates the minimization of memory stalls during sequential access. We perform sequential reads starting from an unaligned address (e.g., $Addr = 247$). We expect one compulsory miss followed by n hits within the same 32-byte block.

4.2 Associativity and LRU Eviction

Tests the structural limits of *Set 7*. We engineer a trace to access nine unique addresses $\{A_1, A_2, \dots, A_9\}$ such that:

$$\text{index}(A_i) = 7, \quad \text{tag}(A_i) \neq \text{tag}(A_j)$$

This forces an eviction of the LRU victim upon the 9th access.

4.3 Integrated Application Stress Test

Simulates a realistic software workload by combining three behaviors:

- **Sequential Processing:** Writing to a 1024-byte array (Spatial Locality).
- **Temporal Re-use:** Repeated access to "hot" loop variables.
- **Conflict Stress:** Inducing a "dirty" eviction by exceeding set associativity.

5 Results and Analysis

5.1 Analysis of Spatial Locality

The simulation logs confirm the 100-cycle DRAM penalty.

- **Latency:** Initial miss at $t = 1$ ns completed at $t = 101$ ns.
- **Hit Performance:** Subsequent reads within the range $[224, 255]$ completed in 1 cycle.
- **Boundary Detection:** Accessing $Addr = 256$ correctly triggered a miss as the index shifted to *Set 8*.

5.2 Analysis of Associativity and LRU

The cache successfully populated ways 0–7 within *Set 7*. Upon requesting the 9th tag at 825 ns, the cache logged the eviction of $Addr = 224$ (the oldest entry). This confirms the LRU algorithm's efficacy.

5.3 Analysis of Application Stress Test

The simulation metrics confirm the cache effectively optimizes complex patterns:

- **Overall Hit Rate (88.77%):** Demonstrates that the 32 KiB structure captures the application's working set.
- **Read Hit Rate (96.08%):** High efficiency due to spatial locality during array processing.
- **Write Hit Rate (87.58%):** Confirms **Allocate-on-Write** logic; after initial line fills, subsequent writes hit the SRAM.
- **Write-Back Penalty:** The total time (5297 ns) includes 100-cycle stalls for writing "dirty" victims back to DRAM.

Appendix: Simulation Logs

Experiment 1 (Spatial Locality)

```
1 ns:      cpu: read on address 247
1 ns:      cache: read address = 247
1 ns:      cache: read miss address = 247
1 ns:      memory: received read on address 247
101 ns:     cache: read completed address = 247 set = 7 line = 0
103 ns:     cpu: read done address = 247
104 ns:     cpu: read on address 224
104 ns:     cache: read address = 224
104 ns:     cache: read hit address = 224 set = 7 line = 0
106 ns:     cpu: read done address = 224
... [Sequential Hits 225 to 255 omitted for brevity] ...
197 ns:     cpu: read on address 255
197 ns:     cache: read address = 255
197 ns:     cache: read hit address = 255 set = 7 line = 0
199 ns:     cpu: read done address = 255
200 ns:     cpu: read on address 256
200 ns:     cache: read address = 256
200 ns:     cache: read miss address = 256
200 ns:     memory: received read on address 256
300 ns:     cache: read completed address = 256 set = 8 line = 0
302 ns:     cpu: read done address = 256
303 ns:     cpu: read on address 222
303 ns:     cache: read address = 222
303 ns:     cache: read miss address = 222
303 ns:     memory: received read on address 222
403 ns:     cache: read completed address = 222 set = 6 line = 0
405 ns:     cpu: read done address = 222
```

Final Statistics:

Reads: 35 | RHit: 32 | RMiss: 3 | RHitrate: 91.43%

Total simulation time: 407 ns

Experiment 2 (Least Recently Used)

```
1 ns:      cpu: read on address 224
1 ns:      cache: read address = 224
1 ns:      cache: read miss address = 224
1 ns:      memory: received read on address 224
101 ns:     cache: read completed address = 224 set = 7 line = 0
103 ns:     cpu: read done address = 224
104 ns:     cpu: read on address 4320
104 ns:     cache: read address = 4320
104 ns:     cache: read miss address = 4320
104 ns:     memory: received read on address 4320
204 ns:     cache: read completed address = 4320 set = 7 line = 1
206 ns:     cpu: read done address = 4320
207 ns:     cpu: read on address 8416
207 ns:     cache: read address = 8416
207 ns:     cache: read miss address = 8416
207 ns:     memory: received read on address 8416
307 ns:     cache: read completed address = 8416 set = 7 line = 2
309 ns:     cpu: read done address = 8416
310 ns:     cpu: read on address 12512
310 ns:     cache: read address = 12512
310 ns:     cache: read miss address = 12512
```

```

310 ns:      memory: received read on address 12512
410 ns:      cache: read completed address = 12512 set = 7 line = 3
412 ns:      cpu: read done address = 12512
413 ns:      cpu: read on address 16608
413 ns:      cache: read address = 16608
413 ns:      cache: read miss address = 16608
413 ns:      memory: received read on address 16608
513 ns:      cache: read completed address = 16608 set = 7 line = 4
515 ns:      cpu: read done address = 16608
516 ns:      cpu: read on address 20704
516 ns:      cache: read address = 20704
516 ns:      cache: read miss address = 20704
516 ns:      memory: received read on address 20704
616 ns:      cache: read completed address = 20704 set = 7 line = 5
618 ns:      cpu: read done address = 20704
619 ns:      cpu: read on address 24800
619 ns:      cache: read address = 24800
619 ns:      cache: read miss address = 24800
619 ns:      memory: received read on address 24800
719 ns:      cache: read completed address = 24800 set = 7 line = 6
721 ns:      cpu: read done address = 24800
722 ns:      cpu: read on address 28896
722 ns:      cache: read address = 28896
722 ns:      cache: read miss address = 28896
722 ns:      memory: received read on address 28896
822 ns:      cache: read completed address = 28896 set = 7 line = 7
824 ns:      cpu: read done address = 28896
825 ns:      cpu: read on address 32992
825 ns:      cache: read address = 32992
825 ns:      cache: read miss address = 32992
825 ns:      cache: evict clean line evict_addr = 224 set = 7 line = 0 tag = 0
825 ns:      memory: received read on address 32992
925 ns:      cache: read completed address = 32992 set = 7 line = 0
927 ns:      cpu: read done address = 32992

```

Final Statistics:

Reads: 9 | RHit: 0 | RMiss: 9 | Hitrate: 0%

Total simulation time: 929 ns

Experiment 3 (Simulation Results) The following table summarizes the cumulative performance metrics for the Integrated Application Stress Test:

CPU	Reads	RHit	RMiss	Writes	WHit	WMiss	RHit%	WHit%	Total Hit%
0	51	49	2	314	275	39	96.08	87.58	88.77

Total simulation time: 5297 ns